DSA Using C Programming - MCQ Questions (Paper Set-1)

::Q1::Which data structure follows the LIFO (Last In First Out) principle?{ =Stack ~Queue ~Linked List ~Tree } // Answer: Stack

---

::Q2::Which data structure follows the FIFO (First In First Out) principle?{ =Queue ~Stack ~Tree ~Graph } // Answer: Queue

---

::Q3::Which operation is used to insert an element into a stack?{ =Push ~Pop ~Peek ~Delete } // Answer: Push

---

::Q4::Which operation removes the top element from a stack?{ =Pop ~Push ~Peek ~Insert } // Answer: Pop

---

::Q5::Which operation displays the top element without removing it?{ =Peek ~Push ~Pop ~Delete } // Answer: Peek

---

::Q6::What is the initial value of top in an ascending stack?{ =-1 ~0 ~1 ~SIZE } // Answer: -1

::Q7::What is the initial value of top in a descending stack?{ =SIZE ~-1 ~0 ~1 } // Answer: SIZE

::Q8::Which condition indicates Stack Overflow?{ =When the stack is full ~When the stack is empty ~When the queue is full ~When memory allocation fails } // Answer: When the stack is full

---

::Q9::Which condition indicates Stack Underflow?{ =When the stack is empty ~When the stack is full ~When the queue is full ~When memory allocation fails } // Answer: When the stack is empty

---

::Q10::Which pointer is used to insert an element into a queue?{ =Rear ~Front ~Top ~Head } // Answer: Rear

---

::Q11::Which pointer is used to delete an element from a queue?{ =Front ~Rear ~Top ~Tail } // Answer: Front

---

::Q12::Which data structure is used internally for recursion?{ =Stack ~Queue ~Tree ~Graph } // Answer: Stack

---

::Q13::Which function is used to allocate dynamic memory in C?{ =malloc() ~printf() ~scanf() ~strlen() } // Answer: malloc()

::Q14::Which function is used to release dynamically allocated memory?{ =free() ~malloc() ~calloc() ~realloc() } // Answer: free()

::Q15::Which searching algorithm requires the array to be sorted?{ =Binary Search ~Linear Search ~Sequential Search ~Interpolation Search } // Answer: Binary Search

---

::Q16::What is the worst-case time complexity of Linear Search?{ =O(n) ~O(log n) ~O(1) ~$O(n^2)$ } // Answer: O(n)

::Q17::What is the worst-case time complexity of Binary Search?{ =O(log n) ~O(n) ~$O(n^2)$ ~O(1) } // Answer: O(log n)

::Q18::Which sorting algorithm repeatedly compares adjacent elements?{ =Bubble Sort ~Selection Sort ~Insertion Sort ~Merge Sort } // Answer: Bubble Sort

---

::Q19::Which sorting algorithm repeatedly selects the minimum element?{ =Selection Sort ~Bubble Sort ~Insertion Sort ~Quick Sort } // Answer: Selection Sort

---

::Q20::Which sorting algorithm inserts an element into its correct position in the sorted array?{ =Insertion Sort ~Bubble Sort ~Selection Sort ~Heap Sort } // Answer: Insertion Sort

---

::Q21::Which data structure stores elements in non-contiguous memory locations?{ =Linked List ~Array ~Stack ~Queue } // Answer: Linked List

---

::Q22::What is the first node of a linked list called?{ =Head ~Tail ~Root ~Top } // Answer: Head

---

::Q23::Which linked list contains pointers to both previous and next nodes?{ =Doubly Linked List ~Singly Linked List ~Circular Linked List ~Array List } // Answer: Doubly Linked List

---

::Q24::Which tree traversal follows the order Root → Left → Right?{ =Preorder ~Inorder ~Postorder ~Level Order } // Answer: Preorder

---

::Q25::Which tree traversal follows the order Left → Root → Right?{ =Inorder ~Preorder ~Postorder ~Level Order } // Answer: Inorder

---

::Q26::Which tree traversal follows the order Left → Right → Root?{ =Postorder ~Preorder ~Inorder ~Level Order } // Answer: Postorder

::Q27::Which data structure is commonly used to implement a Priority Queue?{ =Heap ~Stack ~Queue ~Linked List } // Answer: Heap

---

::Q28::Which notation is used to represent algorithm efficiency?{ =Big O Notation ~ASCII ~Unicode ~Binary } // Answer: Big O Notation

---

::Q29::Which tree traversal uses a Queue internally?{ =Level Order Traversal ~Preorder ~Inorder ~Postorder } // Answer: Level Order Traversal

---

::Q30::Which data structure is commonly used to implement the Undo operation?{ =Stack ~Queue ~Tree ~Graph } // Answer: Stack